

OPT Project 2

<Group 5>

미래자동차공학과 2019048440 최윤석

전기공학전공 2020045441 문재욱





- ❖ 서론(Knapsack 문제에 대한 개요) 및 문제 해결을 위한 조건&수치에 대한 정의
- ❖ 동적 프로그래밍(Dynamic Programming)
- ❖ 분기한정법(Branch and Bound)
- ❖ 탐욕 알고리즘(Greedy Algorithm)
- ❖ LP Relaxation, LP Round Down
- ❖ Matlab intlinprog
- ❖ 결과 분석(각 코드별 결과 및 최적해 대소 비교) 및 고찰



- ❖ Knapsack 문제: 제한된 자원 내에서 최대의 이익을 추구하는 조합 최적화 문제
- ❖ Problem1: 동적 프로그래밍(Dynamic Programming)을 활용하여 최적 비용뿐만 아니라 해당 최적 해를 구성하는 아이템 조합(최적 전략)까지 도출할 수 있는 알고리즘을 구현
- ❖ Problem2: Branch and Bound 기법을 학습하여, 탐색 공간을 체계적으로 축소하며 최적 해를 탐색하는 방식의 장단점을 파악
- ❖ 탐욕 알고리즘(Greedy), 선형계획법(Linear Programming, LP), LP 해의 반내림 기반 알고리즘(LP Round Down)과도 비교



p_j, w_j and c are positive integers,

$$\sum_{j=1}^n w_j > c,$$

$w_j \leq c$ for $j \in N$.

$$\frac{p_1}{w_1} \geq \frac{p_2}{w_2} \geq \dots \geq \frac{p_n}{w_n}.$$

1. 각 물건의 무게와 가치, 가방의 용량은 양의 정수로 정의된다.
2. 각 물건의 무게 합은 가방의 용량보다 커야 한다.
3. 각 물건의 무게는 가방의 용량보다 클 수 없다.
4. 각 물건의 무게와 가치 값에 대해서, 가치/무게 값이 큰 순서부터 나열해야 한다.

문제 해결에 필요한 수치 정의



```
import numpy as np
from scipy.optimize import linprog
import time

np.random.seed(15)

# 1. 설정 (값들은 모두 정수를 가지도록 설정)
size = 10 # 아이템 수
W = np.random.randint(5, 50) # 배낭 최대 무게

# 적절히 작은 무게로 제한 (1 ~ W//3)
while True:
    # 무게가 W를 넘지 않도록 설정
    weights = np.random.randint(1, W // 3, size=size)
    # 무게의 총 합은 W를 초과하도록 설정
    if weights.sum() > W and all(w < W for w in weights):
        break

# 가치도 동일한 사이즈로 생성
values = np.random.randint(5, 10, size=size)

# 출력
print("weights =", list(weights))
print("values =", list(values))
print("capacity =", W) # 가방에 들어갈 수 있는 무게
```

```
weights = [2, 1, 2, 1, 1, 2, 3, 2, 2, 2]
values = [5, 7, 9, 6, 8, 9, 9, 5, 7, 6]
capacity = 13
```

- 1) 시드를 활용하여 랜덤 변수들을 고정한다.
- 2) 물건의 개수를 정의하고, 배낭 최대 무게를 설정한다.
- 3) While 반복문을 통해 가방의 최대 무게를 넘지 않는 물건 무게를 설정하고, 무게의 총 합이 가방 최대 무게를 넘을 때까지 반복한다.
- 4) 가치도 랜덤 값으로 지정한다.



문제 해결에 필요한 수치 정의



```
# 2. 아이템 정보 결합 (무게, 가치, 원래 인덱스 1부터 시작)
items = list(zip(weights, values, range(1, len(weights) + 1)))
```

```
# 3. 정렬 (가치/무게 비율 기준, 내림차순)
n = len(items)
for i in range(n):
    # 한 요소에 대해서 모든 값들과 대소비교하여 자리 조정
    for j in range(0, n - i - 1):
        ratio1 = items[j][1] / items[j][0]
        ratio2 = items[j + 1][1] / items[j + 1][0]
        if ratio1 < ratio2:
            items[j], items[j + 1] = items[j + 1], items[j]
```

```
print("\n[정렬된 아이템 목록] (번호, 무게, 가치):")
for idx, (w, v, original_index) in enumerate(items, start=1):
    print(f"{idx}. ({w}, {v})")
```

[정렬된 아이템 목록] (번호, 무게, 가치):

1. (1, 8)
2. (1, 7)
3. (1, 6)
4. (2, 9)
5. (2, 9)
6. (2, 7)
7. (3, 9)
8. (2, 6)
9. (2, 5)
10. (2, 5)

List를 활용하여 앞서 할당된 무게와 가치의 값들을 연동한다.

이 과정은 각 물건의 무게 정보와 가치 정보를 동시에 파악할 수 있도록 한다.

이후 조건 4)를 만족시키기 위해 각 물건을 가치/무게 크기가 큰 순서대로 나열한다.

For 문을 사용하여 '정렬된 아이템 목록'에 요소를 추가할 때마다 모든 요소와의 대소 비교를 통해 위치를 스위칭한다.

이 때 list를 통해 가치와 무게의 위치를 연동시켰으므로 두 가지 요소의 위치가 동시에 이동한다.



❖ 정의: 문제를 여러 개의 더 간단한 하위 문제로 나누어 해결하는 알고리즘

❖ Greedy 알고리즘과의 비교

- 1) Greedy 알고리즘과 달리 모든 하위 문제를 재귀적으로 해결해야 하므로 더 복잡하고 시간 복잡도가 느리며 이전 하위 문제 해를 저장하는 데 더 많은 메모리를 필요로 함.
- 2) Greedy 알고리즘은 문제에 따라 최적의 해를 얻을 수 있다고 보장할 수 없는 반면, 동적 프로그래밍은 최적해(Optimal Solution)를 확실하게 계산할 수 있다는 장점이 있음.

동적 프로그래밍(Dynamic Programming)



```
242 # 4. DP를 이용한 0/1 Knapsack 알고리즘
243 def Dynamic():
244     start_time=time.time()
245     global items
246     dp = [[0] * (W + 1) for _ in range(n + 1)] # dp[i][w] = i번째까지 고려, 무게 w일 때 최대 가치
247
248     for i in range(1, n + 1):
249         weight = items[i - 1][0]
250         value = items[i - 1][1]
251         for w in range(W + 1):
252             if weight <= w:
253                 dp[i][w] = max(dp[i - 1][w], dp[i - 1][w - weight] + value)
254             else:
255                 dp[i][w] = dp[i - 1][w]
```

(1) DP를 이용한 0/1 Knapsack 알고리즘

1) start_time = time.time() : 함수 시작 시간 기록. 전체 알고리즘의 실행 시간을 측정하는 데 활용.

2) global items: 전역 변수 'items'를 사용하겠다고 선언하는 것. items는 (weight, value)쌍으로 구성된 아이템 리스트임.

3) dp = [[0] * (W + 1) for _ in range(n + 1)]: DP 테이블 초기화. 'dp[i][w]'는 i번째 아이템까지 고려했을 때, 무게 한도가 w일 때 얻을 수 있는 최대 가치를 저장함.

4) for i in range(1, n + 1): 1번 아이템부터 n번 아이템까지 반복. 각 아이템에 대해 DP 값을 계산함.

5) weight = items[i - 1][0]

value = items[i - 1][1]

i번째 아이템의 무게와 가치를 추출

동적 프로그래밍(Dynamic Programming)



```
242 # 4. DP를 이용한 0/1 Knapsack 알고리즘
243 def Dynamic():
244     start_time=time.time()
245     global items
246     dp = [[0] * (W + 1) for _ in range(n + 1)] # dp[i][w] = i번째까지 고려, 무게 w일 때 최대 가치
247
248     for i in range(1, n + 1):
249         weight = items[i - 1][0]
250         value = items[i - 1][1]
251         for w in range(W + 1):
252             if weight <= w:
253                 dp[i][w] = max(dp[i - 1][w], dp[i - 1][w - weight] + value)
254             else:
255                 dp[i][w] = dp[i - 1][w]
```

6) for w in range(W + 1): 배낭의 용량 w를 0부터 W까지 반복하며 해당 무게에서의 최대 가치를 계산

7) if weight <= w: 현재 아이템의 무게가 w보다 작거나 같다면, 해당 아이템을 넣을 수 있는 경우

8) $dp[i][w] = \max(dp[i - 1][w], dp[i - 1][w - \text{weight}] + \text{value})$

: (1) 현재 아이템을 넣지 않았을 때의 최대 가치 $dp[i - 1][w]$ 와 (2) 현재 아이템을 넣었을 때의 최대 가치 $dp[i - 1][w - \text{weight}] + \text{value}$ 중 더 큰 값을 선택하여 $dp[i][w]$ 에 저장

9) else:

$dp[i][w] = dp[i - 1][w]$

현재 아이템을 넣을 수 없는 경우, 이전까지의 최적값을 그대로 가져옴.





```
257     # 5. 선택된 아이템 역추적 - optimal solution에서 어떤 요소가 선택되었는지 파악 가능
258     selected_items = []
259     w = W
260     for i in range(n, 0, -1):
261         if dp[i][w] != dp[i - 1][w]:
262             selected_items.append(items[i - 1])
263             w -= items[i - 1][0] # 무게만큼 줄이기
```

(2) 선택된 아이템 역추적: 앞서 작성한 dp 테이블을 바탕으로, 최적해를 구성한 아이템 목록을 추적하는 부분. 즉, 어떤 아이템들이 최종적으로 선택되었는지 알아내기 위한 과정

- 1) `selected_items = []`: 최적해에 포함된 아이템들을 저장할 리스트
- 2) `w = W`: 현재 배낭에 남은 무게를 의미함. 역추적의 시작점. 즉, 처음에는 최대 용량 `W`에서 출발하여 선택된 아이템의 무게만큼 줄여가며 추적함.
- 3) `for i in range(n, 0, -1)`: `i`번 아이템부터 1번 아이템까지 역순으로 확인
- 4) `if dp[i][w] != dp[i - 1][w]`: `i`번째 아이템이 선택되었는지 확인하는 조건. 만약 `dp[i][w]`의 값이 `dp[i - 1][w]`와 다르다면, `i`번째 아이템을 넣음으로써 가치가 증가한 것이므로, 선택된 것임.
- 5) `selected_items.append(items[i - 1])`: 선택된 아이템을 `selected_items` 리스트에 추가함.
- 6) `w -= items[i - 1][0]`: 선택된 아이템의 무게만큼 배낭 용량 `w`를 줄임. 이후 아이템을 고려할 때는 이 줄어든 무게를 기준으로 고려함.



```
265 # 6. 선택된 아이템 출력
266 print('\n', '='*50)
267 print('\n[Dynamic Programming 결과]')
268 print("\n선택된 아이템 (정렬 후 번호, 무게, 가치):")
269 C=0
270 for item in reversed(selected_items): # 선택 순서를 앞에서부터 보기 위해 뒤집음
271     index_in_sorted = items.index(item) + 1
272     A={item[0]}
273     D=list(A)[0]
274     C+=D
275     print(f"{index_in_sorted}. ({item[0]}, {item[1]})")
276
277 end_time= time.time() - start_time
278 print('\n걸린 시간', end_time)
279 print("\n최대 무게:", C)
280 print("최대 가치:", dp[n][W])
```

(3) 선택된 아이템 출력

1) for item in reversed(selected_items):

선택된 아이템 리스트를 뒤집어 앞에서부터 보기 위해 작성한 부분. 즉, selected_items는 역추적 과정에서 뒤에서부터 채워졌기 때문에 이를 다시 원래의 순서대로 보기 위한 것.

2) index_in_sorted = items.index(item) + 1

: 원래의 'items' 리스트에서 해당 아이템의 인덱스를 찾아 아이템 번호로 표시. +1을 하는 이유는 출력 시 인덱스가 1번부터 시작하는 것처럼 보이게 하기 위한 것.

3) C += D: 선택된 아이템의 무게를 누적하여 총 선택 무게 계산.

4) print(f"{index_in_sorted}. ({item[0]}, {item[1]})")

: 선택된 아이템의 번호, 무게, 가치를 출력.

동적 프로그래밍(Dynamic Programming)



```
265 # 6. 선택된 아이템 출력
266 print('\n', '*50)
267 print('\n[Dynamic Programming 결과]')
268 print("\n선택된 아이템 (정렬 후 번호, 무게, 가치):")
269 C=0
270 for item in reversed(selected_items): # 선택 순서를 앞에서부터 보기 위해 뒤집음
271     index_in_sorted = items.index(item) + 1
272     A={item[0]}
273     D=list(A)[0]
274     C+=D
275     print(f"{index_in_sorted}. ({item[0]}, {item[1]})")
276
277 end_time= time.time() - start_time
278 print('\n걸린 시간', end_time)
279 print("\n최대 무게:", C)
280 print("최대 가치:", dp[n][W])
```

5) print("\n최대 무게:", C)

: 실제로 배낭에 담은 총 무게를 출력

6) print("최대 가치:", dp[n][W])

: DP 테이블에서 마지막 위치에 있는 최댓값을 출력(최종 정답)

7) end_time = time.time() - start_time :알고리즘 실행 시간 측정

[Dynamic Programming 결과]

선택된 아이템 (정렬 후 번호, 무게, 가치):

```
1. (1, 8)
2. (1, 7)
3. (1, 6)
4. (2, 9)
5. (2, 9)
6. (2, 7)
8. (2, 6)
9. (2, 5)
```

걸린 시간 0.0

최대 무게 : 13

최대 가치 : 57

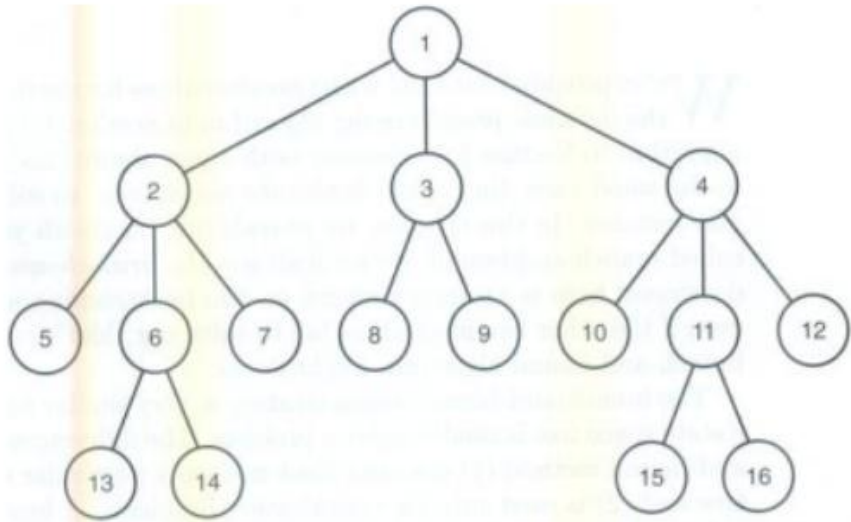
<파이썬 코드 실행 결과>





1. 너비우선탐색 (Breadth-First-search ,BFS)
2. 분기한정법(Branch and Bound) 개념
3. 분기한정법(Branch and Bound) Python 코드분석

너비 우선 탐색(Breadth-First Search, BFS)



- ✓ Breadth-first search (BFS): 블라인드 탐색 방법 중 하나로서, 시작 노드를 방문한 후 시작 노드와 인접한 모든 노드를 우선적으로 방문하는 방식
- ✓ 더 이상 방문하지 않은 노드가 없을 때까지 방문하지 않은 모든 노드에 BFS를 계속 적용.

✓ 장점:

- 시작 노드에서 타겟 노드까지의 최단 경로를 보장함.

✓ 단점:

- 경로가 매우 길면 탐색 가지수가 급격히 늘어나 더 많은 메모리 공간을 필요로 함.
- 해결책이 없을 때, 유한 그래프의 경우 전체 그래프를 탐색한 후 탐색은 결과적으로 실패로 끝남.
- 무한 그래프의 경우, 해를 찾지도 못하고 종료되지도 못함.

분기한정법(Branch and Bound) 개념



- 1) 트리 탐색 방법에 의해 제한되지 않음.
- 2) 최적화 문제를 해결하는 데에만 활용됨.

분기 한정법(Branch and Bound)에서는 back-tracking 외에도 어떤 노드가 promise한지 판단하기 위해 경계값(bound value)를 계산한다. 이 경계값(bound value)이 지금까지 찾은 최적해보다 좋으면 promise한 것으로 본다. 분기 한정법(Branch and Bound)은 back-tracking 알고리즘과 마찬가지로 일반적으로 최악의 경우에 지수적 시간 복잡도를 보인다. 하지만 실제 상황에서는 매우 효율적인 것으로 밝혀졌다.

$n = 4, W = 160$ 이고,

i	p_i	w_i	$\frac{p_i}{w_i}$
1	\$40	2	\$20
2	\$30	5	\$6
3	\$50	10	\$5
4	\$10	5	\$2

일 때,

$$totweight = weight + \sum_{j=i+1}^{k-1} w_j$$

$$bound = \left(profit + \sum_{j=i+1}^{k-1} p_j \right) + (W - totweight) \times \frac{p_k}{w_k}$$

Branch and Bound Algorithm(Python)



```
106 # ----- Branch and Bound 알고리즘 구현 -----
107 def Branch_and_Bound():
108     start_time = time.time()
109     global items
110     # 상태 노드 클래스 정의
111     class Node:
112         def __init__(self, level, value, weight, bound, selected):
113             self.level = level          # 현재 고려 중인 아이템 인덱스
114             self.value = value          # 현재까지 누적된 가치
115             self.weight = weight        # 현재까지 누적된 무게
116             self.bound = bound          # 이 노드에서 기대할 수 있는 최대 가치 상한
117             self.selected = selected    # 현재까지 선택한 아이템 인덱스 목록
118
119     # bound(상한값) 계산 함수
120     def bound(node):
121         if node.weight >= W:
122             return 0 # 무게 초과 시 상한은 0
123
124         profit_bound = node.value # 현재 가치에서 시작
125         j = node.level           # 다음 아이템부터 탐색
126         totweight = node.weight
127
128         # 배낭이 넘치지 않는 선까지 아이템을 greedy하게 추가
129         while j < n and totweight + items[j][0] <= W:
130             totweight += items[j][0]
131             profit_bound += items[j][1]
132             j += 1
133
134         # 그 다음 아이템을 분할하여 부분적으로 넣기 (fractional knapsack 방식)
135         if j < n:
136             weight, value, _ = items[j]
137             profit_bound += (W - totweight) * (value / weight)
138
139         return profit_bound
140
```

<class& bound함수 정의>

<초기화 및 탐색 과정>

```
132 # 초기화
133 max_value = 0          # 지금까지 발견된 최대 가치
134 best_selection = []    # 최적 선택 목록 (인덱스 리스트)
135
136 queue = [] # 노드를 담을 큐
137
138 # 루트 노드 생성 및 큐에 추가
139 root = Node(level=0, value=0, weight=0, bound=bound(Node(0, 0, 0, 0, [])), selected=[])
140 queue.append(root)
141
142 # Branch and Bound 탐색 시작
143 while queue:
144     # 큐에서 bound가 가장 큰 노드 선택 (최대한 가능성 높은 노드)
145     max_idx = 0
146     for i in range(1, len(queue)):
147         if queue[i].bound > queue[max_idx].bound:
148             max_idx = i
149     node = queue.pop(max_idx)
150
151     # 가지치기 조건: bound가 최대값보다 작거나, 모든 아이템을 다 본 경우
152     if node.bound <= max_value or node.level >= n:
153         continue
154
155     # 현재 아이템 정보 가져오기
156     weight, value, _ = items[node.level]
```



Branch and Bound Algorithm(Python)



```
158 # 왼쪽 자식 노드: 현재 아이템을 선택한 경우
159 if node.weight + weight <= W:
160     left = Node(
161         level=node.level + 1,
162         value=node.value + value,
163         weight=node.weight + weight,
164         bound=0,
165         selected=node.selected + [node.level]
166     )
167     left.bound = bound(left) # bound 갱신
168
169 # 더 나은 해를 발견하면 저장
170 if left.value > max_value:
171     max_value = left.value
172     best_selection = left.selected
173
174 # 여전히 탐색 가치가 있으면 큐에 추가
175 if left.bound > max_value:
176     queue.append(left)
177
178 # 오른쪽 자식 노드: 현재 아이템을 선택하지 않은 경우
179 right = Node(
180     level=node.level + 1,
181     value=node.value,
182     weight=node.weight,
183     bound=0,
184     selected=node.selected
185 )
186 right.bound = bound(right)
187
188 if right.bound > max_value:
189     queue.append(right)
```

왼쪽 자식 노드& 오른쪽
자식 노드 생성 및 각각
의 bound 다시 계산

```
191 # 선택된 아이템들의 총 무게 계산
192 total_weight = sum(items[idx][0] for idx in best_selection)
193
194 # 결과 출력
195 print('\n', '='*50)
196 print('\n[Branch and Bound 결과]')
197 print("\n[선택된 아이템들] (정렬된 순서 기준 번호, 무게, 가치):")
198 for idx in best_selection:
199     w, v, _ = items[idx]
200     print(f"{idx + 1}. ({w}, {v})")
201 end_time= time.time() - start_time
202 print('\n걸린 시간', end_time)
203 print(f"\n최대 무게: {total_weight}") # 최대 무게 출력
204 print(f"최대 가치: {max_value}")
```

최종 선택된 아이템들과 실행
시간, 최대 무게 및 가치를 출력





<Branch and Bound 실행결과>

[Branch and Bound 결과]

[선택된 아이템들] (정렬된 순서 기준 번호, 무게, 가치):

1. (1, 8)
2. (1, 7)
3. (1, 6)
4. (2, 9)
5. (2, 9)
6. (2, 7)
8. (2, 6)
9. (2, 5)

걸린 시간 0.0010001659393310547

최대 무게 : 13

최대 가치 : 57

<Dynamic Programming 실행결과>

[Dynamic Programming 결과]

선택된 아이템 (정렬 후 번호, 무게, 가치):

1. (1, 8)
2. (1, 7)
3. (1, 6)
4. (2, 9)
5. (2, 9)
6. (2, 7)
8. (2, 6)
9. (2, 5)

걸린 시간 0.0

최대 무게 : 13

최대 가치 : 57

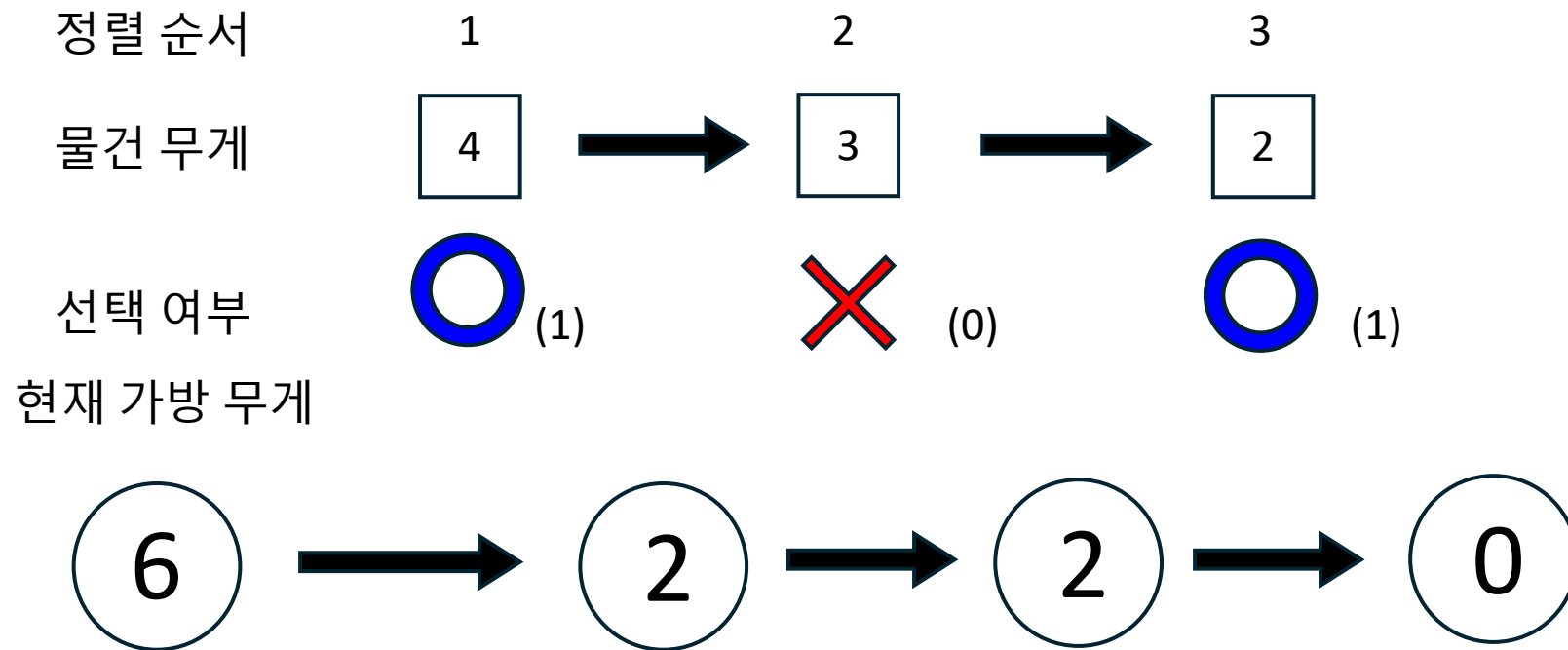
최대 무게와 최대 가치는 동일하게 나왔지만 러닝타임에 살짝 차이가 있음.

Greedy Algorithm



knapsack 문제에서의 Greedy 알고리즘은 단순한 구조를 가진다.

앞서 언급한 조건 성립 하에서 가방에 들어갈 물건을 선택할 때, $\frac{\text{가치}}{\text{무게}}$ 가 큰 순서대로 선택하는 과정이다.



Greedy Algorithm



```
def Greedy():
    start_time=time.time()
    global items
    total_weight = 0
    total_value = 0
    selected_items = []

    # 정렬된 데이터를 앞에서부터 고려하여 선택
    for weight, value, original_index in items:
        if total_weight + weight <= W:
            total_weight += weight
            total_value += value
            selected_items.append((weight, value, original_index))

    # 결과 출력
    print('\n', '='*50)
    print("\n[Greedy Algorithm 결과]")
    print("\n[선택된 아이템들] (번호, (무게, 가치)):")
    for idx, (weight, value, original_index) in enumerate(selected_items, start=1):
        print(f"{idx}. ({weight}, {value})")

    end_time= time.time() - start_time
    print('\n걸린 시간', end_time)
    print(f"\n최종 총 무게: {total_weight}")
    print(f"\n최종 총 가치: {total_value}")
```

Greedy 함수를 정의하고, 총 무게와 총 가치를 계산하기 위한 변수를 지정해준다.

For 문을 이용하여 물건을 넣는 행위를 반복하고, if문을 이용하여 total weight가 가방의 용량인 W를 넘지 않을 때만 물건을 집어넣는 행위를 한다.

물건을 집어넣었다면 total weight와 total value 에 합산하고, selected items 에 선택한 아이템의 정보를 추가한다.

Print 문을 활용해 선택된 아이템의 인덱스 번호, 무게, 가치 순으로 출력하고, 마지막에는 최종 무게와 총 가치를 출력하도록 한다.



```
def Greedy():
    start_time=time.time()
    global items
    total_weight = 0
    total_value = 0
    selected_items = []

    # 정렬된 데이터를 앞에서부터 고려하여 선택
    for weight, value, original_index in items:
        if total_weight + weight <= W:
            total_weight += weight
            total_value += value
            selected_items.append((weight, value, original_index))

    # 결과 출력
    print('\n', '='*50)
    print("\n[Greedy Algorithm 결과]")
    print("\n[선택된 아이템들] (번호, (무게, 가치)):")
    for idx, (weight, value, original_index) in enumerate(selected_items, start=1):
        print(f"{idx}. ({weight}, {value})")

    end_time= time.time() - start_time
    print("\n걸린 시간", end_time)
    print(f"\n최종 총 무게: {total_weight}")
    print(f"\n최종 총 가치: {total_value}")
```

[Greedy Algorithm 결과]

[선택된 아이템들] (번호, (무게, 가치)):

1. (1, 8)
2. (1, 7)
3. (1, 6)
4. (2, 9)
5. (2, 9)
6. (2, 7)
7. (3, 9)

걸린 시간 0.0732419490814209

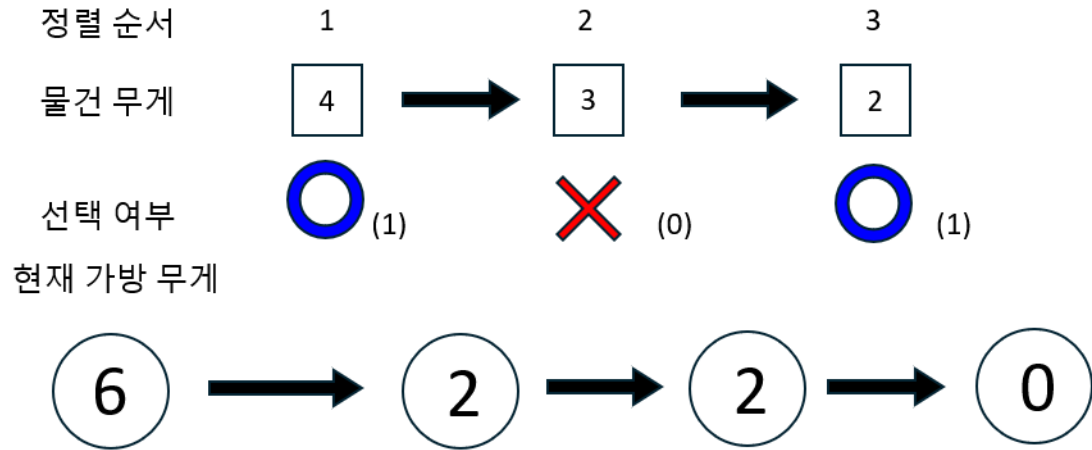
최종 총 무게: 12

최종 총 가치: 55

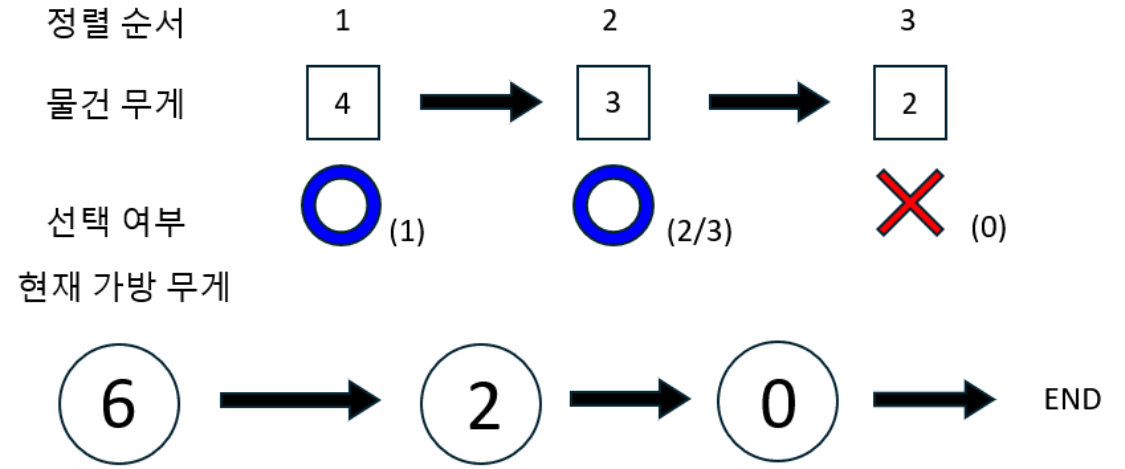
앞서 가치의 가성비가 높은 순서대로 정렬된 물건들을 앞에서부터 고려하여 선택의 과정을 거치고, 결과를 출력하면 다음과 같다.



기존 Knapsack (Greedy 방법)



LP Relaxation



LP Relaxation은 기존의 Knapsack 문제의 조건을 완화시켜 해결하는 방법이다.

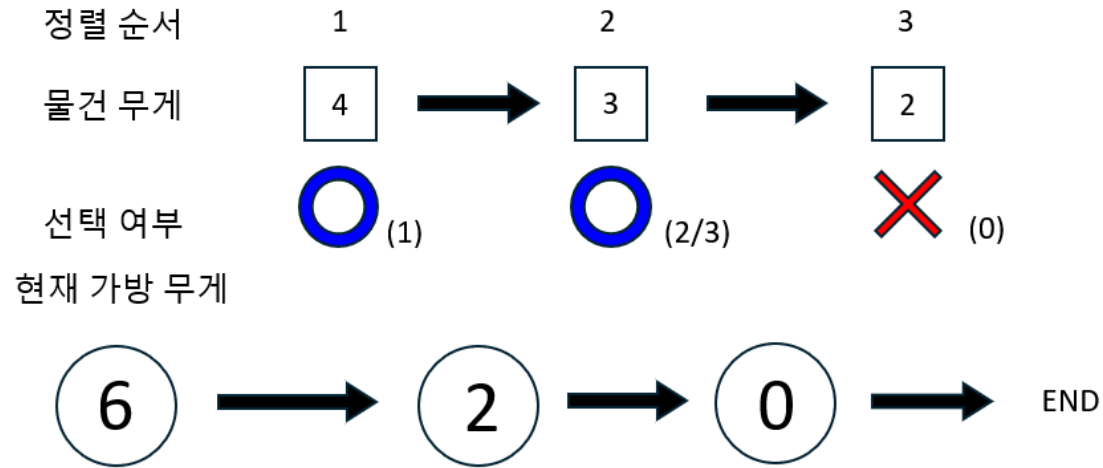
Knapsack 문제에서는 물건을 선택할 때 0과 1의 선택지만 존재하기 때문에 x 가 0과 1인 정수 값만을 가질 수 있었다.

하지만, LP Relaxation을 통해 0과 1사이의 실수를 해로 가질 수 있도록 조건을 완화하여 해를 구한다.

이로 인하여 물건의 선택 여부가 자유로워졌으며, 선택 사항을 고려할 때 비교적 단순한 선택을 할 수 있게 된다.



LP Relaxation



x 값이 0과 1 사이의 소수 값을 가질 수 있기 때문에 가치의 가성비 (가치/무게)가 큰 물건부터 차례대로 고려하면 되는 것이다.

기존의 Knapsack 문제라면 여러가지 조합을 고려해야 했겠지만, LP Relaxation에서는 큰 물건부터 차례대로 선택하는 것이 가장 큰 이득이 되고, 가방을 완전하게 다 채울 수 있다.



```
# 목표: maximize sum(values[i] * x[i])
# linprog는 기본이 minimization이므로, -values로 변경해서 최대화 문제로 변환
def LP_Relaxation():
    start_time = time.time()
    global items
    c = [-item[1] for item in items] # 가치를 최대화하기 위해 부호 반전
    A = [[item[0] for item in items]] # 무게에 대한 제약 조건
    b = [W] # 총 무게가 W 이하
    x_bounds = [(0, 1) for _ in items] # 각 아이템의 선택 비율 범위 (0~1)

    # linprog 호출
    res = linprog(c, A_ub=A, b_ub=b, bounds=x_bounds, method='highs')

    # 결과 출력
    print('Wn', '='*50)
    print("Wn[LP Relaxation 결과]")
    if res.success:
        total_value = -res.fun # 최적화 결과에서 가치
        # 총 무게 계산
        total_weight = sum(res.x[idx] * items[idx][0] for idx in range(len(res.x)))

        print("아이템별 선택 확률 (순서대로):")
        for idx, x in enumerate(res.x, start=1):
            w, v, original_index = items[idx - 1]
            print(f"item {idx}: 선택 비율 = {x:.4f}") # 아이템 선택 확률 출력
        end_time1 = time.time() - start_time
        print('Wn결린 시간', end_time1)
        print(f"Wn최종 총 무게: {total_weight:.2f}")
        print(f"Wn최종 총 가치 (fractional): {total_value:.2f}")
    else:
        print("최적화 실패:", res.message)
```

[LP Relaxation 결과]
아이템별 선택 확률 (순서대로):
item 1: 선택 비율 = 1.0000
item 2: 선택 비율 = 1.0000
item 3: 선택 비율 = 1.0000
item 4: 선택 비율 = 1.0000
item 5: 선택 비율 = 1.0000
item 6: 선택 비율 = 1.0000
item 7: 선택 비율 = 0.6667
item 8: 선택 비율 = 1.0000
item 9: 선택 비율 = 0.0000
item 10: 선택 비율 = 0.0000

결린 시간 0.1520547866821289

최종 총 무게: 13.00
최종 총 가치 (fractional): 58.00

x의 바운드를 0과 1사이의 값으로 지정해주었고, linprog를 호출하여 문제를 해결하였다.

이 때, 출력 결과에서 item 7의 값이 아이템 8의 값보다 작은 것을 확인할 수 있다.

이론대로라면 인덱스의 오름차순으로 값이 선택되고, 마지막 값은 소수 값이 나타나야 한다.

이러한 이유는 처음 지정된 무게와 가치 값 때문이다.

LP Relaxation + Round Down



[LP Relaxation 결과]

아이템별 선택 확률 (순서대로):

item 1: 선택 비율 = 1.0000
item 2: 선택 비율 = 1.0000
item 3: 선택 비율 = 1.0000
item 4: 선택 비율 = 1.0000
item 5: 선택 비율 = 1.0000
item 6: 선택 비율 = 1.0000
item 7: 선택 비율 = 0.6667
item 8: 선택 비율 = 1.0000
item 9: 선택 비율 = 0.0000
item 10: 선택 비율 = 0.0000

걸린 시간 0.1520547866821289

최종 총 무게: 13.00

최종 총 가치 (fractional): 58.00

[정렬된 아이템 목록] (번호, 무게, 가치):

1. (1, 8)
2. (1, 7)
3. (1, 6)
4. (2, 9)
5. (2, 9)
6. (2, 7)
7. (3, 9)
8. (2, 6)
9. (2, 5)
10. (2, 5)

7번과 8번을 보면, (가치/무게) 값이

$(9/3)=(6/2)=3$ 으로 같기 때문에, 문제가 되지 않는다.

(단, 이러한 상황은 Greedy에서 추가 정렬 규칙이 요구된다.)

5. LP 결과를 Round Down 하기

```
rounded_x = np.floor(total_value) # 소수점 절삭
print('Wn', '='*50)
print("\n[Round Down 결과]")
print(f"Wn최종 총 무게:", total_weight)
print("최종 총 가치 (fractional):", rounded_x)
```

[Round Down 결과]

최종 총 무게: 13.0

최종 총 가치 (fractional): 58.0

추가로, Round Down 을 수행하여 최종 가치의 소수점을 제거할 수도 있다.

기존 LP 결과는 소수점이 없는 결과가 도출되었지만, 결론 및 고찰에서 추가 상황을 탐색해본다.



```
% 문제 데이터
weights = [1, 1, 1, 2, 2, 2, 3, 2, 2, 2]; % 아이템 무게
values = [8, 7, 6, 9, 9, 7, 9, 6, 5, 5]; % 아이템 가치
W = 13; % 배낭의 최대 무게

n = length(weights); % 아이템 개수

% intlinprog 기본 형식은 "minimize" 이므로, maximize를 위해 부호를 반대로
f = -values; % maximize value -> minimize (-value)
intcon = 1:n; % 모든 변수를 정수형 (binary)
A = weights; % 무게 제약
b = W; % 최대 무게

% intlinprog 실행
opts = optimoptions('intlinprog', 'Display', 'off');
[x, fval] = intlinprog(f, intcon, A, b, [], [], zeros(n,1), ones(n,1), opts);

% 선택된 아이템 출력
disp(['[선택된 아이템들] (정렬된 순서 기준 번호, 무게, 가치):']);
selected_items = find(x == 1); % 선택된 아이템들의 인덱스

% 선택된 아이템을 번호순으로 정렬하여 출력
for i = 1:length(selected_items)
    idx = selected_items(i);
    fprintf('%d. (%d, %d)\n', idx, weights(idx), values(idx));
end

% 최대 무게와 최대 가치 출력
total_weight = sum(weights(selected_items)); % 선택된 아이템들의 총 무게
total_value = -fval; % 최대 가치 (최소화한 값의 부호 반전)
disp(['최대 무게: ', num2str(total_weight)]);
disp(['최대 가치: ', num2str(total_value)]);
```

[선택된 아이템들] (정렬된 순서 기준 번호, 무게, 가치):

```
1. (1, 8)
2. (1, 7)
3. (1, 6)
4. (2, 9)
5. (2, 9)
6. (2, 7)
8. (2, 6)
10. (2, 5)
최대 무게: 13
최대 가치: 57
```

앞서 살펴본 다양한 방법들에 더해서, 정확한 솔루션을 구하기 위해 매트랩을 사용하여 결과를 확인해보았다.

파이썬에서 정의된 랜덤 값을 매트랩에서 정의한 후, intlinprog 함수를 이용하여 결과를 출력하였다. (크기별 나열 과정이 완료된 값을 입력하였다.)

Intlinprog는 해가 되는 x값을 정수로만 도출하는 함수로, 기존의 Knapsack문제를 해결할 수 있다.



지금까지 구한 Solution들을 비교해보자.

[Dynamic Programming 결과]

[선택된 아이템들] (번호, (무게, 가치)):

1. (1, 8)
2. (1, 7)
3. (1, 6)
4. (2, 9)
5. (2, 9)
6. (2, 7)
8. (2, 6)
9. (2, 5)

걸린 시간 0.08648514747619629

최대 무게: 13
최대 가치: 57

[Branch and Bound 결과]

[선택된 아이템들] (번호, (무게, 가치)):

1. (1, 8)
2. (1, 7)
3. (1, 6)
4. (2, 9)
5. (2, 9)
6. (2, 7)
8. (2, 6)
9. (2, 5)

걸린 시간 0.05836319923400879

최대 무게: 13
최대 가치: 57

Dynamic Programming과 Branch and Bound 방법은 모든 상황을 전부 고려하는 방법으로, 가장 최적화된 해를 제공한다. 이는 앞선 Matlab inlinprog에서도 확인할 수 있다.





[Greedy Algorithm 결과]

[선택된 아이템들] (번호, 무게, 가치):

1. (1, 8)
2. (1, 7)
3. (1, 6)
4. (2, 9)
5. (2, 9)
6. (2, 7)
7. (3, 9)

걸린 시간 0.07737946510314941

최종 총 무게: 12

최종 총 가치: 55

[LP Relaxation 결과]

아이템별 선택 확률 (순서대로):

- item 1: 선택 비율 = 1.0000
- item 2: 선택 비율 = 1.0000
- item 3: 선택 비율 = 1.0000
- item 4: 선택 비율 = 1.0000
- item 5: 선택 비율 = 1.0000
- item 6: 선택 비율 = 1.0000
- item 7: 선택 비율 = 0.6667
- item 8: 선택 비율 = 1.0000
- item 9: 선택 비율 = 0.0000
- item 10: 선택 비율 = 0.0000

걸린 시간 0.1520547866821289

최종 총 무게: 13.00

최종 총 가치 (fractional): 58.00

[Round Down 결과]

최종 총 무게: 13.0

최종 총 가치 (fractional): 58.0

Greedy 알고리즘은 선택 방법을 단순화하여 결과를 도출하는 방법으로, Dynamic과 Branch and Bound 보다는 작은 가치가 도출되었다. 이는 그리디 알고리즘 특성 상 가방을 전부 채우지 못하는 상황이 발생하기 때문이다.

LP Relaxation은 물건을 선택하는 상황을 다소 비현실적으로 가정하지만, 가방의 무게 용량을 항상 꼭 채울 수 있으며, 가장 높은 가치를 지니는 상황을 도출하게 만들어준다. 따라서 Dynamic, Branch and Bound 방법보다 큰 가치를 도출할 수 있다.

이에 더해서 Round Down 방법은 LP Relaxation의 비현실적인 가정을 바로잡기 위해 결과를 정수로 취하고, 이 과정에서 소수점 버림이 발생한다. 이로 인해 LP Relaxation 보다는 작은 가치를 지닌다.



4가지 방법을 통해 얻은 결과를 정리하면 다음과 같다.

$\text{Greedy} \leq \text{Dynamic} = \text{Branch and Bound} \leq \text{LP Relaxation Round down} \leq \text{LP Relaxation}$

$(55 \leq 57 \leq 58 \leq 58)$

이에 더해서, 추가로 몇 가지 상황을 살펴보자.



[정렬된 아이템 목록] (번호, 무게, 가치):

1. (3, 13)
2. (4, 12)
3. (5, 11)
4. (7, 13)
5. (8, 12)
6. (11, 13)
7. (10, 11)
8. (13, 14)
9. (15, 14)
10. (13, 12)

[Dynamic Programming 결과]

최대 무게: 48
최대 가치: 85

=====

[Branch and Bound 결과]

최대 무게: 48
최대 가치: 85

=====

[Greedy Algorithm 결과]

최종 총 무게: 48
최종 총 가치: 85

=====

[LP Relaxation 결과]

최종 총 무게: 48.00
최종 총 가치 (fractional): 85.00

=====

[Round Down 결과]

최종 총 무게: 48
최종 총 가치: 85

위와 같은 상황에서는 5가지의 결과가 모두 같은 값을 가졌다.

Greedy=Dynamic=Branch and Bound=LP Relaxation Round down=LP Relaxation





[정렬된 아이템 목록] (번호, 무게, 가치):

1. (1, 14)
2. (1, 12)
3. (1, 11)
4. (2, 11)
5. (3, 14)
6. (3, 13)
7. (3, 13)
8. (3, 13)
9. (4, 12)
10. (4, 12)

[Dynamic Programming 결과]

최대 무게: 15
최대 가치: 90

=====

[Branch and Bound 결과]

최대 무게: 15
최대 가치: 90

[Greedy Algorithm 결과]

최종 총 무게: 14
최종 총 가치: 88

=====

[LP Relaxation 결과]

최종 총 무게: 16.00
최종 총 가치 (fractional): 96.67

=====

[Round Down 결과]

최종 총 무게: 16.0
최종 총 가치 (fractional): 96.0

위와 같은 상황에서는 5가지의 LP Relaxation 의 Round Down 과정에서 소수점 버림이 발생하였다.

$\text{Greedy} \leq \text{Dynamic} = \text{Branch and Bound} \leq \text{LP Relaxation Round down} < \text{LP Relaxation}$



감사합니다